

Laboratori de GIA-ABIA

Sessió 5: Cerca local (simulated annealing)

En la sessió anterior vam veure la resolució del bin packing amb *hill climbing*, en aquesta veurem la manera de resoldre'l amb *simulated annealing*.

Implementant els elements del problema

De moment, conservarem les mateixes implementacions dels següents elements:

- Configuració del problema: `bin_packing_problem_parameters.py`
- Operadors: `bin_packing_operators.py`
- Problema: `bin_packing_problem.py`
- Representació de l'estat: `bin_packing_state.py`. **Atenció! Torneu-vos a baixar aquest fitxer del web de l'assignatura perquè hem trobat un error.**

Per utilitzar *simulated annealing*, el que farem serà importar la funció de `search.py` que implementa l'algoritme i després procedirem a canviar la classe de representació de l'estat:

```
1 from aima.search import hill_climbing, simulated_annealing
2 from bin_packing_problem_parameters import ProblemParameters
3 from bin_packing_problem import BinPackingProblem
4 from bin_packing_state import generate_initial_state
5
6 params = ProblemParameters(3, [1, 1, 1, 1, 1], 5, 5)
7 initial_state = generate_initial_state(params)
8 n = simulated_annealing(BinPackingProblem(initial_state))
9 print(n)                # Estat final
10 print(n.heuristic())    # Valor de l'estat final
```

Anem a comparar el temps d'execució contra el *hill climbing*. Augmentem el tamany del problema i utilitzem la llibreria *timeit* per comparar. Podeu fer un codi similar a aquest:

```
1 import timeit
2
3 from aima.search import hill_climbing, simulated_annealing
4 from bin_packing_problem_parameters import ProblemParameters
5 from bin_packing_problem import BinPackingProblem
6 from bin_packing_state import generate_initial_state
7
```

```

8  v_h = [i for i in range(50)]
9  initial_state = generate_initial_state(ProblemParameters(50,
    v_h, 50, 50))
10
11  hc_h = hill_climbing(BinPackingProblem(initial_state))
12  hc_t = timeit.timeit(lambda: hill_climbing(BinPackingProblem(
    initial_state)), number=5)
13
14  sa_h = simulated_annealing(BinPackingProblem(initial_state))
15  sa_t = timeit.timeit(lambda: simulated_annealing(
    BinPackingProblem(initial_state)), number=5)
16
17  print(f"HC, num. contenidors: {hc_h.heuristic()} contenidors")
18  print(f"HC, temps/5 execs. : {hc_t}s.")
19  print(f"SA, num. contenidors: {sa_h.heuristic()} contenidors")
20  print(f"SA, temps/5 execs. : {sa_t}s.")

```

Quins resultats us dona?

En quant a heurística, degut a la naturalesa aleatòria del simulated annealing, pot ser que haguem de fer més proves per saber si tendeix a guanyar a hill climbing o no. Com a exercici de cara a la pràctica, podeu fer més proves i calcular la mitjana. Per buscar una bona qualitat haurem de jugar amb els paràmetres de la funció de refredament del simulated annealing: k (constant proporcional), lambda (exponent) i número d'iteracions. Per exemple:

```

1  sa_h = simulated_annealing(BinPackingProblem(initial_state),
    schedule=exp_schedule(k=1, lam=0.0005, limit=25000))

```

En qualsevol cas, anem a centrar-nos ara en el temps d'execució. Sembla que no guanyem res utilitzant simulated annealing. Ho podem millorar?

Aprofitant l'algoritme de simulated annealing per optimitzar l'execució

Aquí teniu el codi de la funció simulated_annealing tal i com està escrit al fitxer search.py:

```

1  def simulated_annealing(problem, schedule=exp_schedule()):
2      current = Node(problem.initial)
3      for t in range(sys.maxsize):
4          T = schedule(t)
5          if T == 0:
6              return current.state
7          neighbors = current.expand(problem)
8          if not neighbors:
9              return current.state

```

```

10         next_choice = random.choice(neighbors)
11         delta_e = problem.value(next_choice.state) - problem.
            value(current.state)
12         if delta_e > 0 or probability(np.exp(delta_e / T)):
13             current = next_choice

```

Preneu especial atenció a la línia:

```

1 next_choice = random.choice(neighbors)

```

Com s'explica a teoria, una particularitat d'aquest algorisme és que en cada iteració només necessita un successor aleatori. El que hem fet en l'execució anterior era el mateix que hem fet amb altres algorismes: generar tots els successors possibles a partir de tots els operadors aplicables i deixar que l'algorisme escollís un, en base a una heurística. Com podem millorar això?

Ja que tenim el coneixement sobre la representació de l'estat i dels operadors (perquè els hem desenvolupat nosaltres), podem aprofitar per generar un successor de manera aleatòria sense haver de generar els altres. Això sí, ho hem de fer sobre una distribució de probabilitat uniforme en base al conjunt complet de possibles successors, per tal de no esbiaixar uns operadors per sobre d'altres.

Aquest seria el procediment pel problema de Bin Packing:

1. Calcular quantes combinacions de paràmetres són vàlides per MoveParcel en l'estat actual. A aquest número l'anomenarem N.
2. Calcular quantes combinacions de paràmetres són vàlides per SwapParcel en l'estat actual. A aquest número l'anomenarem M.
3. Ara que sabem que hi ha un total d' $N + M$ successors possibles, escollirem un operador. Utilitzarem un generador de números aleatoris per generar un real aleatori X entre 0 i 1. Si X és menor que $N/(N+M)$, escollirem MoveParcel. En cas contrari, escollirem SwapParcel.
4. Per l'operador escollit, calcularem paràmetres aleatòriament tenint en compte les seves restriccions.
5. Retornem una llista únicament amb l'operador i paràmetres escollits. Com que serà l'únic operador generat, serà l'escollit per l'algorisme.

Per tant, seguint aquesta mecànica, a la classe StateRepresentation podem afegir aquesta funció generate_one_action (la teniu al fitxer generate_one_action.py del web de laboratori de l'assignatura):

```

1 def generate_one_action(self) -> Generator[BinPackingOperator,
        None, None]:
2     # Primer calculem l'espai lliure de cada contenidor

```

```

3     free_spaces = []
4     for c_i, parcels in enumerate(self.v_c):
5         h_c_i = self.params.h_max
6         for p_i in parcels:
7             h_c_i = h_c_i - self.params.v_h[p_i]
8         free_spaces.append(h_c_i)
9
10    # Recorregut contenidor per contenidor per saber quins
    paquets podem moure
11    move_parcel_combinations = set()
12    for c_j, parcels in enumerate(self.v_c):
13        for p_i in parcels:
14            for c_k in range(len(self.v_c)):
15                # Condiçió : contenidor diferent i t      espai
                lliure suficient
16                if c_j != c_k and free_spaces[c_k] >= self.
                    params.v_h[p_i]:
17                    move_parcel_combinations.add((p_i, c_j, c_k
                        ))
18
19    # Intercanviar paquets
20    swap_parcel_combinations = set()
21    for p_i in range(self.params.p_max):
22        for p_j in range(self.params.p_max):
23            if p_i != p_j:
24                c_i = self.find_container(p_i)
25                c_j = self.find_container(p_j)
26
27                if c_i != c_j:
28                    h_p_i = self.params.v_h[p_i]
29                    h_p_j = self.params.v_h[p_j]
30
31                    # Condiçió: hi ha espai lliure suficient
                    per fer l'intercanvi
32                    # (Espai lliure del contenidor + espai que
                    deixa el paquet >= espai del nou paquet)
33                    if free_spaces[c_i] + h_p_i >= h_p_j and
                        free_spaces[c_j] + h_p_j >= h_p_i:
34                        swap_parcel_combinations.add((p_i, p_j
                            ))
35
36    n = len(move_parcel_combinations)
37    m = len(swap_parcel_combinations)
38    random_value = random.random()
39    if random_value < (n / (n + m)):
40        combination = random.choice(list(
            move_parcel_combinations))
41        yield MoveParcel(combination[0], combination[1],
            combination[2])
42    else:

```

```
43         combination = random.choice(list(
44             swap_parcel_combinations))
        yield SwapParcels(combination[0], combination[1])
```

Presteu especial atenció en el fet de que no estem generant nous objectes per cada combinació, només estem creant un sol operador MoveParcel o SwapParcels quan ja sabem quin operador i combinació volem. Això farà que la crida random.choice que hi ha a dins de la implementació de l'algoritme només hagi d'escollir entre una (i només una) opció, i per tant que l'execució sigui més ràpida.

No obstant, ara tenim dos funcions diferents per generar operadors, una per generar-los tots i una altra per generar només un. Idealment, hauriem de preparar la nostra implementació per a que el hill climbing utilitzi la primera, i que el simulated annealing utilitzi la segona.

Exercicis proposats

- Busqueu una combinació de paràmetres k, lambda i iteracions que faci que el simulated annealing sigui competitiu (en quant a valor heurístic de la sol·lució trobada) contra el hill climbing.
- Afegiu el mètode generate_one_action a la classe de representació de l'estat.
- Feu modificacions per poder llençar experiments indicant si voleu que l'algoritme utilitzi generate_actions o generate_one_action. Per exemple, podeu afegir un argument nou a la constructora del problema (e.g. use_one_action, i que aquest argument es tingui en compte en el mètode actions()) per escollir quin mètode de l'estat es crida.
- Compareu el rendiment del hill climbing, el simulated annealing generant tots els successors, i el simulated annealing generant només un successor. Es nota el canvi?
- Apliqueu el simulated annealing amb el problema de bin packing optimitzat que us hem passat pel Racó. Com que la representació de l'estat canvia, haureu de fer canvis en generate_one_action. Compareu diferents algoritmes, amb diferents heurístiques, i amb les dos estratègies de generació de sol·lució inicial.